

CSS定位与布局

2025年9月26日

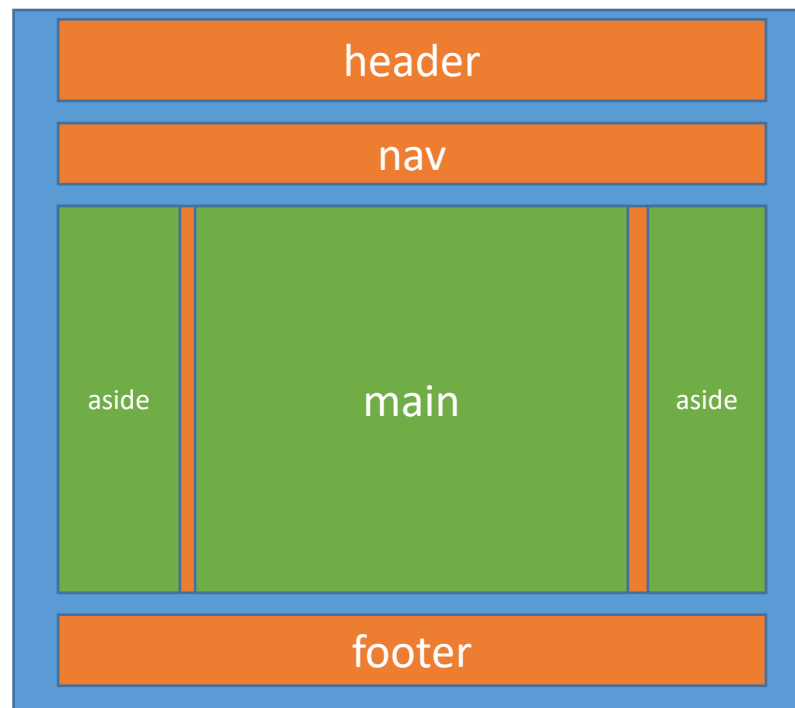
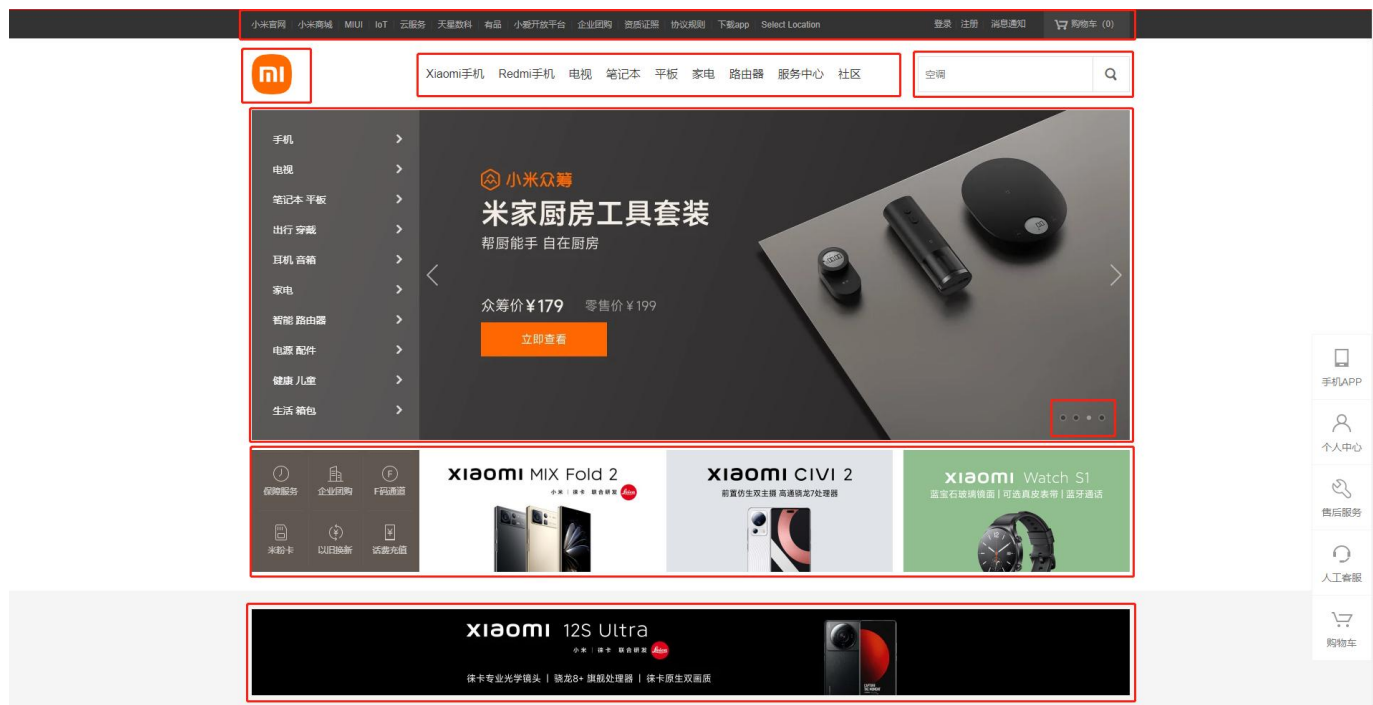
目 录

01 ▶ 盒子模型

02 ▶ CSS定位

01 盒子模型

CSS 盒子模型是网页设计中的基础概念。在CSS中，每个元素都可以看作是一个矩形的盒子，这个盒子包含了元素的内容、内边距、边框和外边距。页面布局就是通过大盒子嵌套小盒子，逐步细化完成。完成元素定位和页面布局后，才涉及具体的设置元素的字体、文本等样式。

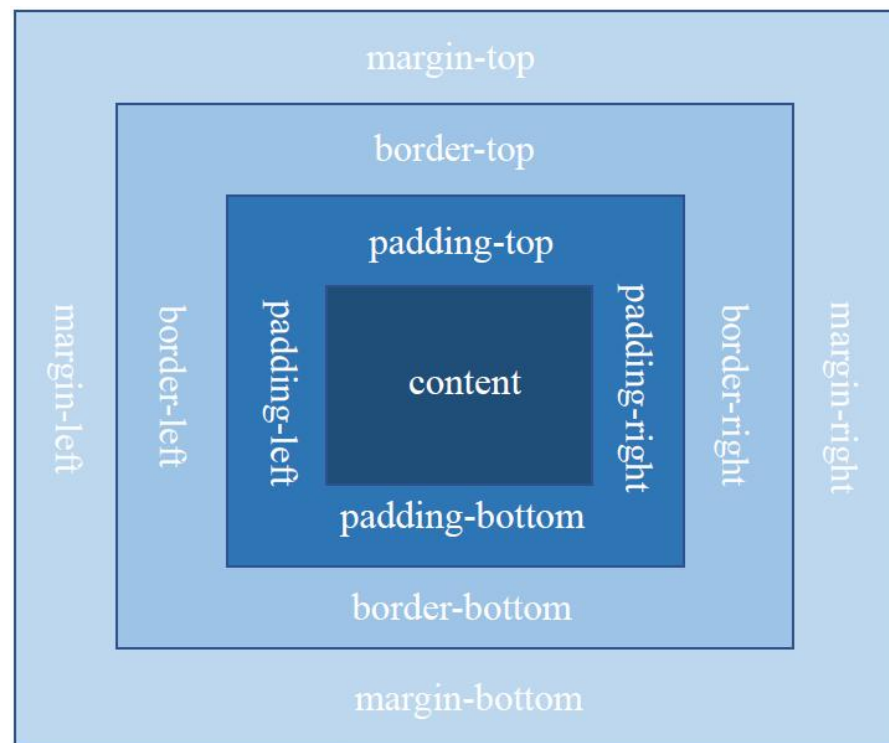


页面上一些区域、图片、导航、列表、段落等等，都可以是盒子

01 盒子模型

CSS 盒子模型包括：边框、内边距、外边距和实际内容。

- content：这个区域是用来显示内容，大小可以通过设置 width 和 height.
- padding: 包围在内容区域外部的空白区域；大小通过 padding 相关属性设置。
- border: 边框盒包裹内容和内边距。大小通过 border 相关属性设置。
- margin: 这是最外面的区域，是盒子和其他元素之间的空白区域。大小通过 margin 相关属性设置。

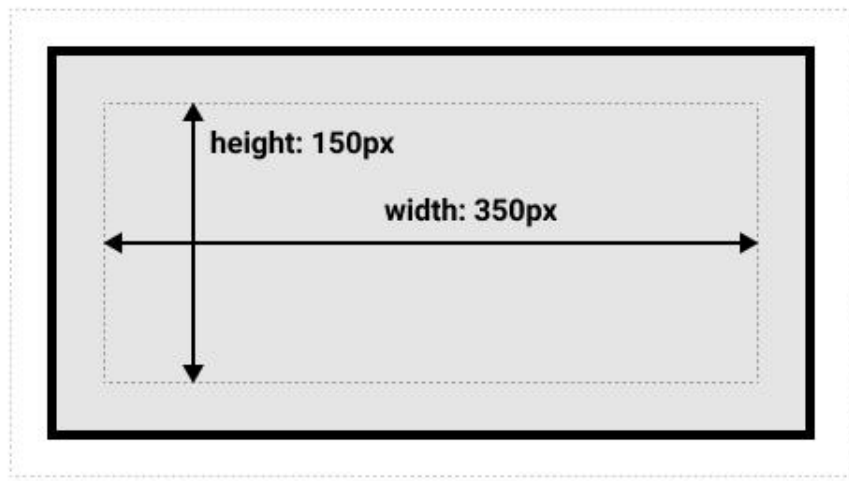


01 盒子模型

盒模型有两种模式：content-box和border-box。

浏览器默认使用的是content-box。padding 和 border 再加上设置的宽（width）高（height）一起决定整个盒子的大小。

```
.box {  
  width: 350px;  
  height: 150px;  
  margin: 25px;  
  padding: 25px;  
  border: 5px solid black;  
}
```



- 如果使用content-box，那么实际占用空间的宽高分别为：宽度 = 410px ($350 + 25 + 25 + 5 + 5$)，高度 = 210px ($150 + 25 + 25 + 5 + 5$)。
- margin 不计入实际大小 —— 但它会影响盒子在页面所占空间，影响的是盒子外部空间。盒子的范围到边框为止 —— 不会延伸到 margin。

01 盒子模型

1、**box-sizing**属性 决定了盒子大小的计算方式:

① 默认取值为**content-box**, 此时width、height指定元素内容的宽度和高度, border和padding不计算入width和height之内。

盒子的总宽度=width+padding+border

盒子的总高度=height+padding+border

以默认的content-box盒子进行页面布局时, 如果已经定义好width和height, 一旦添加border和padding, 都会使盒子实际尺寸变大, 影响布局。

② 另一个取值为**border-box**, 通过**box-sizing: border-box** 设置。

盒子的width、height包含content和padding、border。使用这个模型, 所有宽度都是可见宽度, 所以元素内容宽度是width减去border和padding部分。



01 盒子模型

padding内边距

padding 属性用于设置内边距，内边距位于边框和内容区域之间。与外边距不同，不能设置负值。

属性	作用
padding-left	左内边距
padding-right	右内边距
padding-top	上内边距
padding-bottom	下内边距

padding 属性（简写属性）可以有一到四个值

值的个数	表达意思
padding: 5px;	1个值，代表上下左右都有5像素内边距;
padding: 5px 10px;	2个值，代表上下内边距是5像素 左右内边距是10像素;
padding: 5px 10px 20px;	3个值，代表上内边距5像素 左右内边距10像素 下内边距20像素;
padding: 5px 10px 20px 30px;	4个值， 上是5像素 右10像素 下20像素 左是30像素 顺时针

给盒子指定 padding 值之后：

- 1、内容和边框之间有了距离。
- 2、padding影响了盒子实际大小。

01 盒子模型

练习：padding内边距

padding内边距可以撑开盒子。

因为每个导航栏里面的字数不一样多，可以不用给每个盒子设定宽度，直接添加padding最合适。



相关取值：

1. 上边框为3像素，颜色为 #ff8500
2. 下边框为1像素，颜色为 #edeef0
3. nav盒子高度为45像素，背景颜色为 #fcfcfc
4. 文字颜色为 #4c4c4c

01 盒子模型

border边框

有四个边框，每个边框都有宽度、样式和颜色。可以使用border属性一次设置所有四个边框的宽度、颜色和样式。

border: border-width border-style border-color

分别设置每边的宽度、样式和颜色，可以使用：

- border-top: 1px solid red; */* 只设定上边框， 其余同理 */*
- border-bottom: 1px solid red;
- border-left: 1px solid red;
- border-right: 1px solid red;

也可以单独设置每一边框的某一个属性，比如border-right-color:red;

01 盒子模型

margin外边距

margin 属性用于设置外边距，外边距是盒子周围一圈看不到的空间。它会把其他元素从盒子旁边推开。外边距属性值可以为正也可以为负。设置负值会导致和其他内容重叠。

属性	作用
margin-left	左外边距
margin-right	右外边距
margin-top	上外边距
margin-bottom	下外边距

margin 简写方式代表的意义跟 padding 完全一致

01 盒子模型

margin外边距

外边距可以让块级盒子**水平居中**，但是必须满足两个条件：

1. 盒子必须指定了宽度（width）。
2. 盒子**左右的外边距**都设置为 auto。

```
.header{ width:960px; margin:0 auto;}
```

常见的写法，以下三种都可以：

- margin-left: auto; margin-right: auto;
- margin: auto;
- **margin: 0 auto;**

以上方法是让块级元素水平居中

行内元素或者行内块元素水平居中给其父元素添加 text-align: center ;即可。

01 盒子模型

margin外边距

使用 margin 定义块元素的垂直外边距时，可能会出现外边距合并。主要有两种情况：

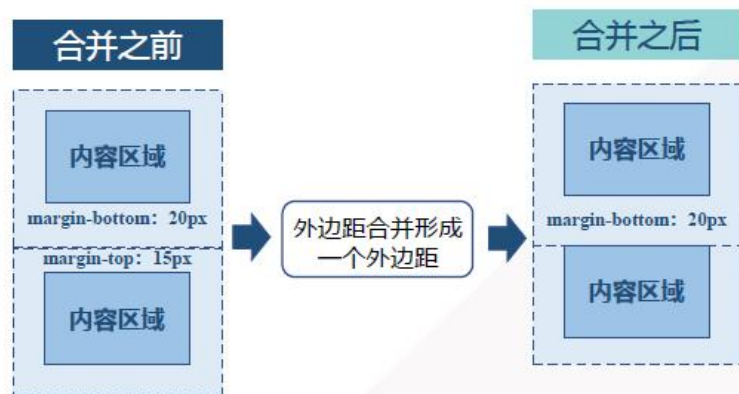
1. 相邻块元素垂直外边距的合并

2. 嵌套块元素垂直外边距的塌陷

注意：垂直方向上相邻的元素才会有外边距合并的问题。

(1). 相邻块元素垂直外边距的合并

当上下相邻的两个块元素（兄弟关系）相遇时，如果上面的元素有下外边距 margin-bottom，下面的元素有上外边距 margin-top，则它们之间的垂直间距不是 margin-bottom 与 margin-top 之和。而是取两个值中的较大者。这种现象称为相邻块元素垂直外边距的合并。



解决方案：

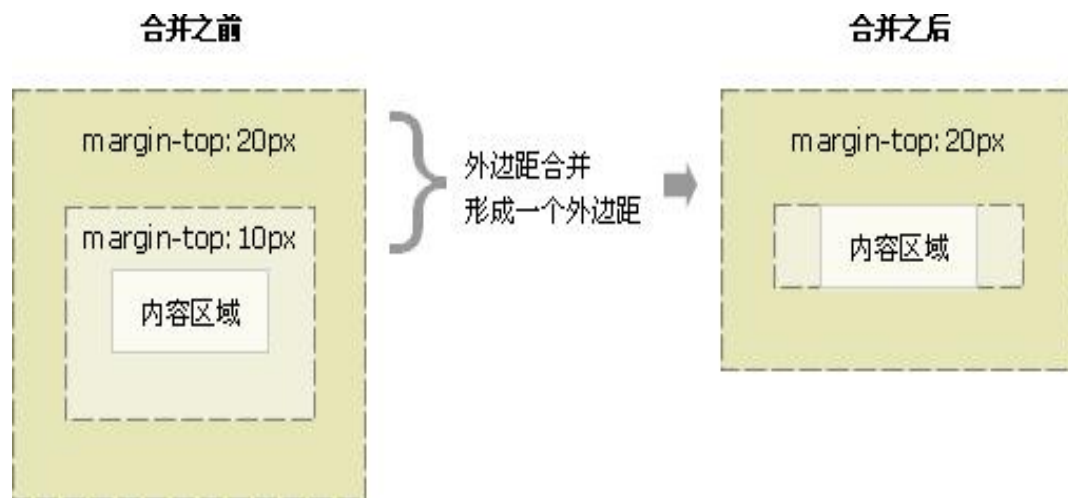
尽量只给一个盒子添加 margin 值。

01 盒子模型

margin外边距

(2).嵌套块元素垂直外边距的塌陷

对于两个具有父子关系的块元素，父元素有上外边距，同时子元素也有上外边距，此时父元素会塌陷相对较大的外边距值，而子元素没有实现上外边距效果。



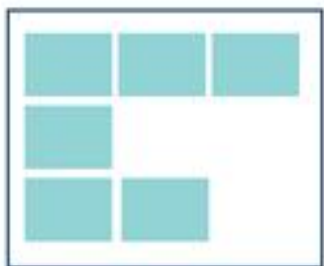
解决方案:

- ① 可以为父元素定义上边框border-top。
- ② 更改子元素的 margin-top 为父元素的 padding-top，同时为防止父元素变大，再设置盒子模型为box-sizing:border-box。
- ③ 可以为父元素设置overflow:hidden。
- ④ 给父或子元素添加**固定定位**或者**绝对定位**。
- ⑤ 给父元素或子元素设置display: inline-block。

02 CSS定位

CSS中的定位机制是用来控制元素在页面上的位置的。**文档流定位**、**浮动定位**和**层定位**是其中三种主要定位方式。

文档流定位



浮动定位



层定位



02 CSS定位

文档流定位是默认的定位方式，它遵循正常的HTML文档流。块级元素会从上到下排列，行内元素则会在块级元素内部从左到右排列。

1、**块级元素 (block)**，会独占一行，从上向下顺序排列。width 和 height 属性可以发挥作用。

内边距 (padding)，外边距 (margin) 和 边框 (border) 会将其他元素从当前盒子周围“推开”。

常用元素：div、hr、p、h1~h6、ul、ol、dl、form、table

2、**行内元素 (inline)**，不会产生换行，从左到右顺序排列，碰到父元素边缘则自动换行。width 和 height 属性不起作用。

常用元素：span、a、i、em、strong等

- 将inline元素显示为block元素，设置：`a{display: block;}`，则inline元素 a 转换为了block 元素，从而使链接元素具有块状元素特点
- 将block 元素显示为inline元素：设置：`div{display:inline;}`

02 CSS定位

3、inline-block元素

同时具备inline 元素、 block 元素的特点

元素同一行显示，除非内容到达父元素边缘，否则不会换行

而元素的height、width、margin、padding都可以设置

最典型的的就是元素。属于inline元素，但表现上更倾向于 inline-block。

通过给元素添加display:inline-block;可以设置元素为inline-block元素。

02 CSS定位

浮动定位是一种传统的布局技术，主要用于实现文本环绕图片效果，也可以用来进行简单的布局。在浮动定位中，元素会脱离文档流，并向左或向右移动，直到它的外边缘碰到包含框或另一个浮动元素的边缘。

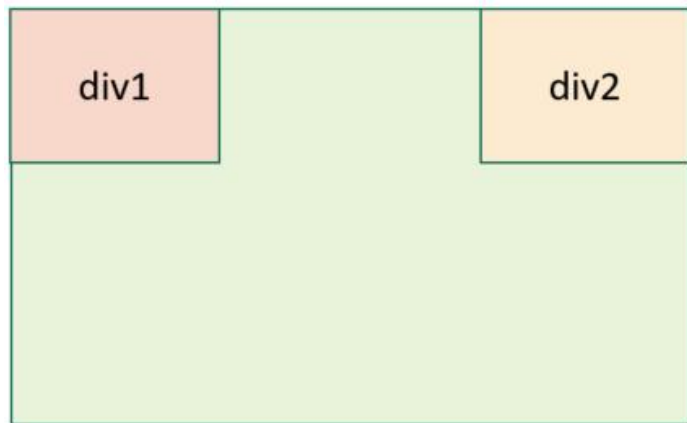
思考：用标准文档流能很方便的实现如下效果吗？

1. 如何让多个块级元素(div)水平排列成一行？



比较难，虽然转换为行内块元素可以实现一行显示，但是他们之间会有大的空白缝隙，很难控制。

2. 如何实现两个盒子的左右对齐？



02 CSS定位

浮动定位

引入 float 属性是为了能让 Web 开发人员实现简单的布局，包括在一列文本中浮动的图像，文字环绕在它的左边或右边。任何东西都可以浮动，不仅仅是图像，所以我们可以看见首字母下沉的应用场景。浮动成为在网页上创建多列布局的最常用方式之一。



Lorem ipsum dolor sit amet consectetur adipisicing elit. Earum, blanditiis possimus? Deserunt dolorem explicabo possimus vel ipsam. Laboriosam ut ex dignissimos tempora. Consectetur id libero doloribus odio distinctio quas vel. Quo, quod. Quos, cum. Nesciunt id quam neque aliquam ex, eius maiores doloribus deserunt cum

cupiditate illo consequatur quisquam inventore nisi, perspiciatis ab modi, nam voluptatibus nobis. Tempore, quidem quos. Tenetur recusandae quam culpa sunt voluptates blanditiis ipsam officia dolorem voluptatem cum? Quaerat sequi autem eaque aut numquam, cumque incidunt odit molestias voluptas doloribus, excepturi vitae minus cupiditate, sapiente nesciunt. Quasi deleniti nesciunt deserunt officia similique voluptatibus atque illo suscipit cum et totam fuga animi autem tempore, saepe sequi rem rerum expedita quaerat veritatis quae optio. Corporis doloremque quas maiores. Tempora minima pariatur architecto eum excepturi vel, voluptate eveniet suscipit itaque veniam totam accusamus beatae? Illum ipsam ex iusto. Reprehenderit iure facilis velit maiores quae incidunt suscipit voluptatibus temporibus obcaecati. Cupiditate atque architecto nam expedita dolores possimus nostrum quam placeat. Commodi illum dolor dignissimos non laboriosam et, magni, eos aperiam repellendus ex consectetur, sint quae perferendis cupiditate tempore aut aliquam. Assumenda nam quis cumque expedita omnis alias atque itaque amet provident, culpa quisquam quidem tempora laudantium voluptatum?

Carol McKinney Highsmith (born Carol Louise McKinney on May 18, 1946) is an American photographer, author, and publisher who has photographed in all the states of the United States, as well as the [District of Columbia](#), and [Puerto Rico](#). She photographs the entire American vista (including landscapes, architecture, urban and rural life, and people in their work environments) in all fifty [U.S. states](#) as a record of the early 21st century.

Highsmith is donating her life's work of more than 100,000 images, royalty-free, to the [Library of Congress](#), which established a rare, one-person archive.

02 CSS定位

浮动定位

主要涉及两个属性，`float`和`clear`。

`float`属性用于设定浮动，创建浮动框，将其移动到一边，直到左边缘或右边缘触及包含块或另一个浮动框的边缘。语法：

选择器 {`float: none | left | right;`}

属性值	描述
<code>none</code>	元素不浮动（默认值）
<code>left</code>	元素向左浮动
<code>right</code>	元素向右浮动

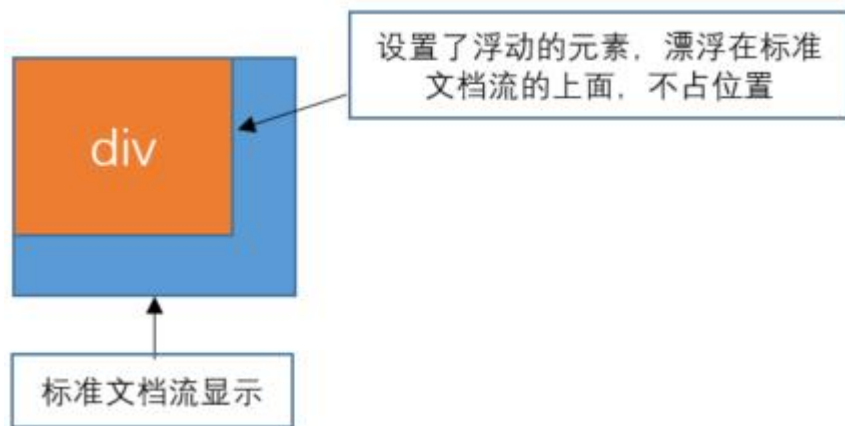
`clear`属性用于清除浮动元素的影响。比如`clear: left;`或`clear: right;`

02 CSS定位

浮动定位

加了浮动之后的元素，会具有很多特性：

1. 浮动元素会脱离标准文档流(脱标)——最重要的一个特性
 2. 浮动的元素会一行内显示并且元素顶部对齐
 3. 浮动的元素会具有行内块元素的特性.
- 浮动的盒子不再保留原先的位置



- 如果多个盒子都设置了浮动，则它们会按照属性值在**一行内显示并且顶端对齐排列**



在一行显示，三个div都要设置浮动属性

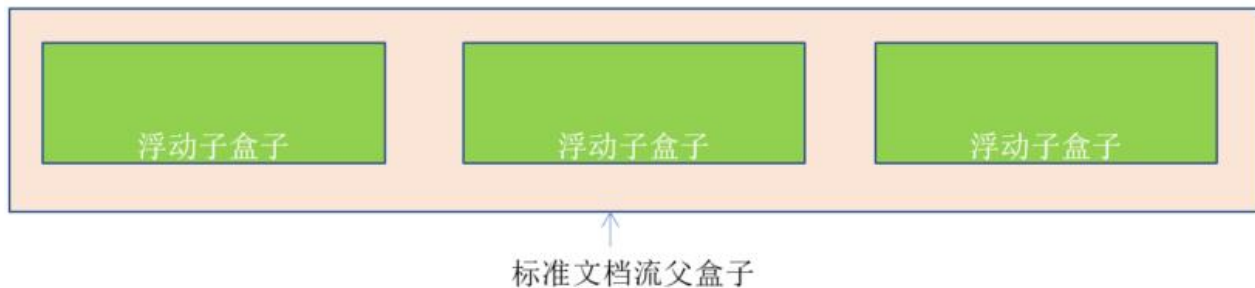
02 CSS定位

浮动定位

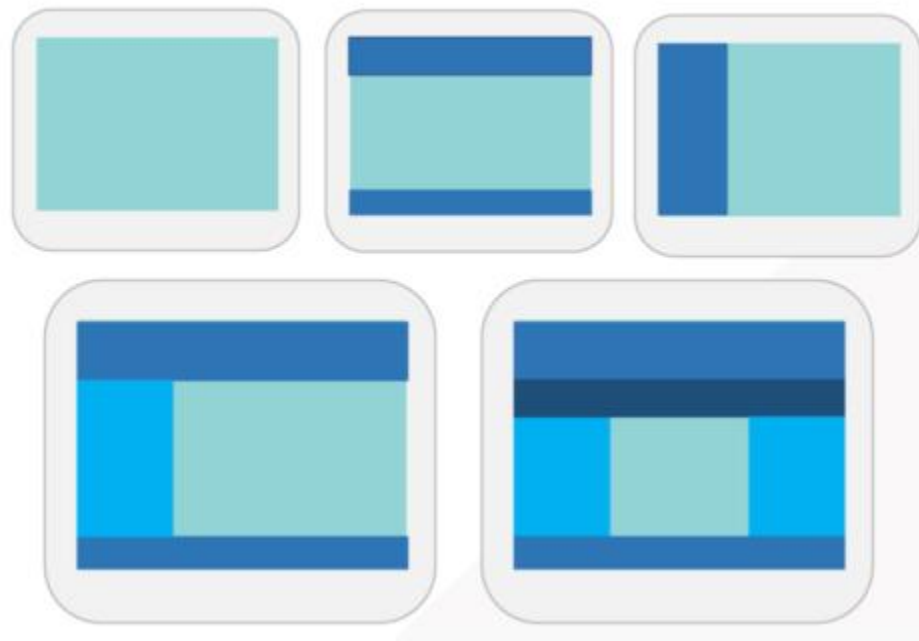
浮动元素经常和标准文档流父级搭配使用。

为了约束浮动元素位置，网页布局一般采取的策略是：

- 先用标准文档流的父元素排列上下位置，
- 之后内部子元素再采取浮动排列左右位置。



常见网页布局



02 CSS定位

层定位

层定位（position属性）允许从正常的文档流布局中取出元素，使元素具有不同的行为，例如放在另一个元素的上面，或者始终保持在浏览器视窗内的固定位置。

应用场景：

- 1、某个元素可以自由的在一个盒子内移动位置，并且压住其他盒子。
- 2、某个盒子可以固定在屏幕中某个位置。

position属性（相对于谁定位）	
static	表示“将元素放入它在文档流定位中的正常位置”，无其他特殊含义
fixed	固定定位
relative	相对定位
absolute	绝对定位
sticky	粘性定位
通过以下属性定位（位置在哪里）	
top right bottom left来精确指定要将定位元素移动到的位置	



02 CSS定位

层定位

一个元素可以自由的在一个盒子内移动位置，并且压住其他盒子。也就是元素开始重叠，那是什么决定哪些元素出现在其他元素的顶部？用**z-index** 属性来控制元素的前后次序 (z轴)。



- 数值可以是正整数、负整数或 0, 默认值是 auto, 数值越大, 元素越靠上
- 如果属性值相同, 则按照书写顺序, 后来居上
- z-index 只接受无单位数值
- 只有层定位的元素才有 z-index 属性

02 CSS定位

层定位

1. 设置静态定位`position: static`

静态定位是元素的默认定位方式，即无定位的意思。

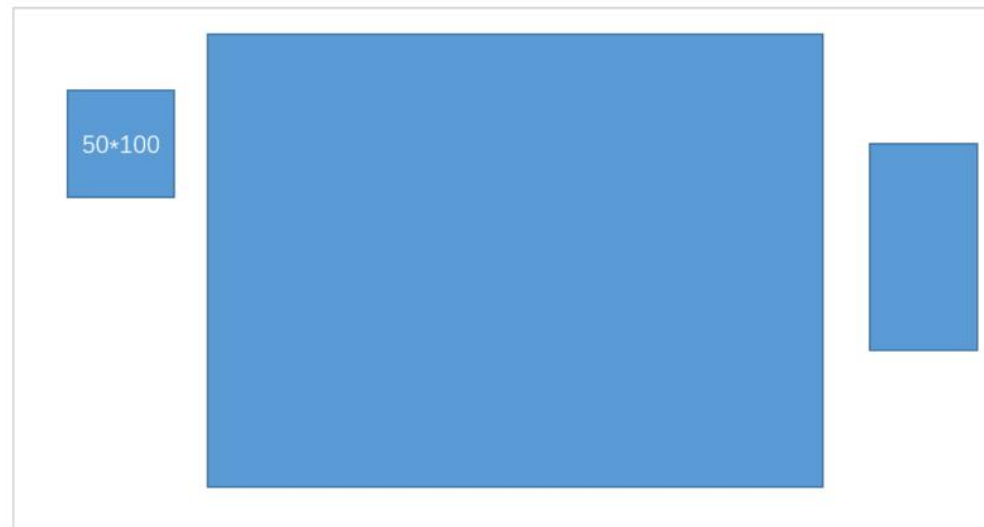
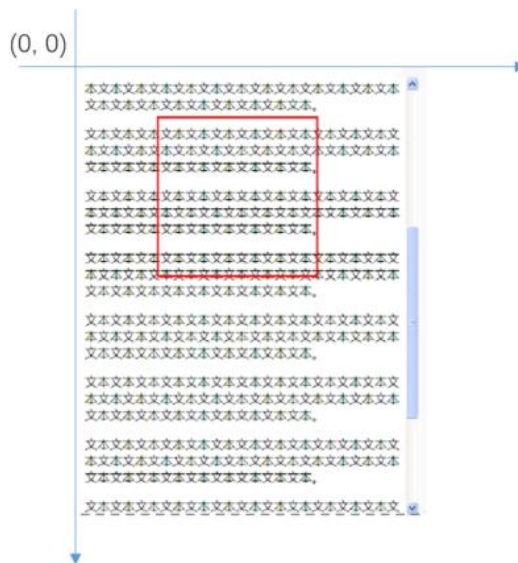
静态定位按照标准文档流特性摆放位置，它没有边偏移，在布局时很少用到。

2. 设置固定定位`position: fixed`

固定定位固定元素是相对于浏览器窗口本身。也就是说我们可以创建固定的有用的 UI 项目，如持久导航菜单。

```
<style>
  .box {
    position: fixed;
    width: 200px;
    height: 200px;
    border: 1px solid red;

    top: 50px;
    left: 100px;
  }
</style>
```

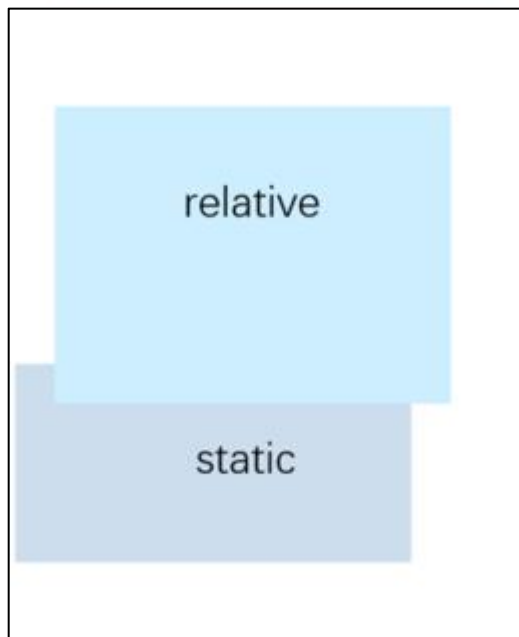


02 CSS定位

层定位

3. 设置相对定位`position:relative`

定位为`relative` 的元素脱离标准文档流，但其在文档流中的**原位置依然存在**
`relative`定位的层总是相对于其**直接父元素**，无论其父元素是什么定位方式。



```
<style>
  .box-relative {
    position: relative;
    top: 20px;
    left: 20px;
    width: 200px;
    height: 150px;
    background-color: #cef;
  }
  .box-static {
    width: 200px;
    height: 100px;
    background-color: #cde;
  }
</style>
</head>
<body>
  <div class="box-relative"></div>
  <div class="box-static"></div>
</body>
```

02 CSS定位

层定位

4. 设置绝对定位`position:absolute`

定位为absolute 的元素的层脱离标准文档流，但与relative 的区别是其在正常文档流中的原位置不再存在



```
<style>
  .box-absolute {
    position: absolute;
    top: 20px;
    left: 20px;
    width: 200px;
    height: 150px;
    background-color: #cef;
  }
  .box-static {
    width: 200px;
    height: 100px;
    background-color: #cde;
  }
</style>
</head>
<body>
  <div class="box-absolute"></div>
  <div class="box-static"></div>
</body>
```

对于absolute定位的层总是相对于其**最近的定义为absolute或relative的父层**，而这个父层并不一定是其直接父层。如果其父层中都未定义absolute或relative，则绝对定位元素根据浏览器视口来定位，即相对于body来进行定位。

02 CSS定位

层定位

5. 设置粘性定位`position: sticky`

粘性定位可以被认为是相对定位(`position: relative`)和固定定位(`position: fixed`)的混合。元素在跨越特定阈值前为相对定位，之后为固定定位。例如：

```
1. .sticky-header {  
2.   position: sticky;  
3.   top: 10px;  
4. }
```

在视口滚动到元素 top 距离小于 10px 之前，元素为相对定位。之后，元素将固定在与顶部距离 10px 的位置，直到视口回滚到阈值以下。

粘性定位常作用在导航和概览信息(标题，表头，操作栏，底部评论等)上。这样，用户在浏览详细信息时，也能看到信息的概览和做一些操作，给用户带来便捷的使用体验。

粘性定位设置很容易出现不生效的情况，具体详见<https://geekdaxue.co/read/fegogogo@fe/hov6gd>

02 CSS定位

层定位

绝对定位的盒子水平居中设置方式:

加了绝对定位的盒子不能再通过 `margin:0 auto;` 水平居中，但是可以通过以下计算方法实现水平居中。

1. `left: 50%;`: 让盒子的左侧移动到父级元素的水平中心位置。
2. `margin-left: -100px;`: 让盒子向左移动自身宽度的一半（假设盒子宽度为200px）。

垂直居中同理:

`top:50%;`

`margin-top:-100px;`

02 CSS定位

层定位

子级元素是绝对定位的话，父级元素要用相对定位

- ① 子级绝对定位，不会占有位置，可以放到父盒子里面的任何一个地方，不会影响其他的兄弟盒子。
- ② 父盒子需要加定位限制子盒子在父盒子内的显示。
- ③ 父盒子在布局时，需要占有位置，因此父盒子只能是相对定位。

相对定位经常用来作为绝对定位的父级。

```
<div class="box1">  
    
  <div class="box2">  
    一起享受咖啡带来的温暖吧  
  </div>  
</div>
```



```
div{  
  border:1px solid red;  
  color: #fff;  
}  
.box1{  
  width:170px;  
  height:190px;  
  position:relative;  
}  
.box2{  
  width:99%;  
  position:absolute;  
  bottom:0;  
}
```

子绝父相

02 CSS定位

CSS 属性建议书顺序：

1. 布局定位属性：display / position / float / clear / visibility / overflow
2. 自身属性：width / height / margin / padding / border / background
3. 文本属性：color / font / text-decoration / text-align / vertical-align / white-space / break-word
4. 其他属性 (CSS3) : content / cursor / border-radius / box-shadow / text-shadow / background:linear-gradient ...

```
.jdc {  
    display: block;  
    position: relative;  
    float: left;  
    width: 100px;  
    height: 100px;  
    margin: 0 10px;  
    padding: 20px 0;  
    font-family: Arial, 'Helvetica Neue', Helvetica, sans-serif;  
    color: #333;  
    background: rgba(0,0,0,.5);  
    border-radius: 10px;  
}
```

02 CSS定位

总结

通过盒子模型，大家知道大部分html标签是一个盒子。

通过CSS浮动、定位属性可以让每个盒子排列布局构成网页。

一个完整的网页，是标准文档流、浮动、层定位一起，依据各自特性完成布局。

1. 标准文档流

可以让盒子上下排列或者左右排列，**垂直的块级盒子显示就用标准文档流布局。**

2. 浮动定位

可以让多个块级元素一行显示或者左右对齐盒子，**多个块级盒子水平显示就用浮动布局。**

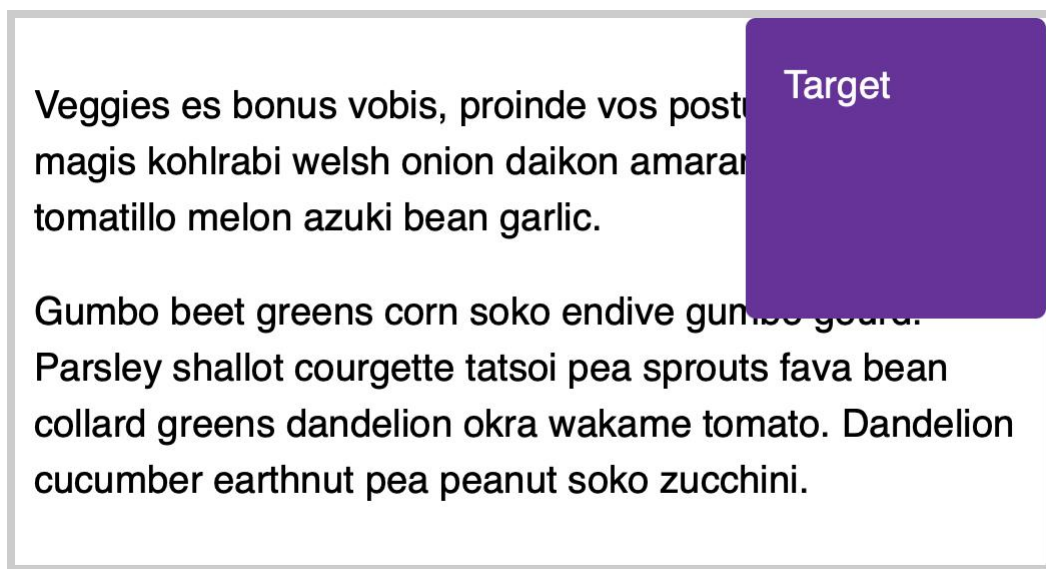
3. 层定位

层定位最大的特点是有层叠的概念，就是可以让多个盒子前后叠压来显示。**如果元素自由在某个盒子内移动就用层定位布局。**

课堂练习

练习一：

将Target紫色矩形元素定位到外部容器的右上角，如图所示。



课堂练习

练习二：

原代码，滚动条滚动时侧边栏将随内容一起滚动。更改CSS使滚动条滚动时侧边栏保持原位并且只滚动内容。
如图所示。

